

Coding & Solution

Date	01 July 2026
Team ID	N/A
Project Name	Personalized Networking Assistant
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/danduthanishkareddy/Personalized-Networking-Assistant
Programming Language(s)	Python,HTML,CSS
Framework(s) Used	Streamlit, FastAPI, Groq API, Wikipedia API, Pydantic, PyTest, HTTPX TestClient

Key Features Implemented	• AI-powered conversation starter generation using Groq API • Personalized self-introduction generation • Smart networking tips generation • Wikipedia-based fact verification • Conversation history management using SQLite (networking.db) • REST APIs using FastAPI • Interactive Streamlit user interface • Automated testing using PyTest
Pending / Incomplete Features	• User authentication and login. • Cloud deployment. • Multilingual support. • Voice-enabled interaction. • LinkedIn profile integration.
Setup / Run Instructions	• Clone the GitHub repository. • Install dependencies using <code>pip install -r requirements.txt</code>. • Create a <code>.env</code> file and add your Groq API key: <code>API_KEY=your_api_key</code>. • Start the FastAPI backend using <code>uvicorn app.main:app --reload</code>. • Launch the Streamlit frontend using <code>streamlit run frontend/app.py</code>. • Open <code>http://localhost:8501</code> in your web browser.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes

4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

- The project follows a modular architecture with separate layers for the Streamlit frontend, FastAPI backend, service modules, API routes, and SQLite database.
- Groq API is integrated to generate personalized conversation starters, self-introductions, and networking tips based on user inputs and event details.
- Wikipedia API is used to verify factual information and improve the reliability of AI-generated networking responses.
- Conversation history is stored and retrieved using the SQLite database (networking.db) to provide persistent local storage.
- FastAPI and Pydantic ensure secure request validation, structured API communication, and high-performance backend processing.
- Streamlit provides an interactive, responsive, and user-friendly interface for students, professionals, and job seekers.
- Automated testing is implemented using pytest and HTTPX TestClient to verify API functionality and improve application reliability.
- The modular architecture makes the application easy to maintain and extend with future enhancements such as authentication, cloud deployment, multilingual support, voice interaction, and LinkedIn integration.